

School Management System

EduSchool - The Next-Gen High School Management System

MENH Chanheng - 2023445

PIN Sovanda - 2023457

HENG Shito - 2023210

INDEX

Serial Number	Topics	Page Number
S01	Introduction	2
S02	Objectives	2
S03	Project Scope	2
S04	Methodology	2
S05	Requirements	14
S06	Timeline	15
S07	Expected Outcomes	15
S08	Challenges and Risks	15
S09	Budget and Resources	16
S10	References	16
S11	Team Member Responsibilities	17

Introduction

- **Problem Statement:** Many high schools still rely on fragmented manual systems or outdated software for record-keeping. This leads to data redundancy, difficulty in generating real-time academic reports [1], and a communication gap between teachers and parents [2].
- **Motivation:** Automating administrative tasks allows educators to focus on teaching rather than paperwork. A centralized system ensures data integrity and provides parents with immediate insights into their child's progress [3].

Objectives

The objectives of our project are:

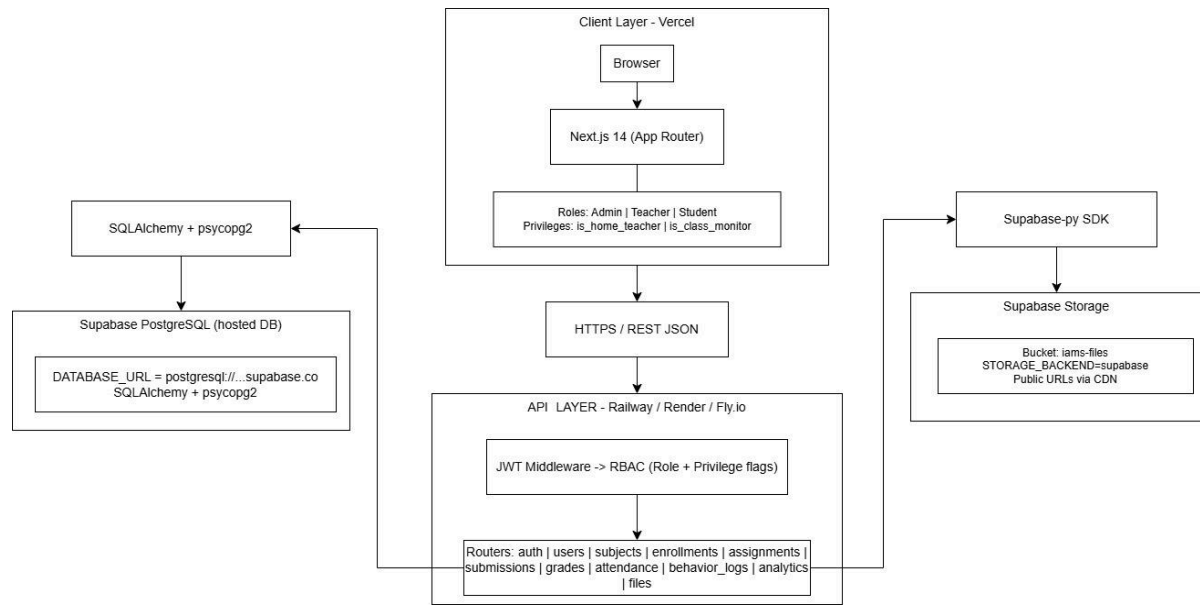
- To develop a centralized database for student, teacher, and staff records.
- To automate grade calculation and report card generation.
- To provide a secure login portal for three distinct user roles: Admin, Teacher, and Student/Parent.

Project Scope

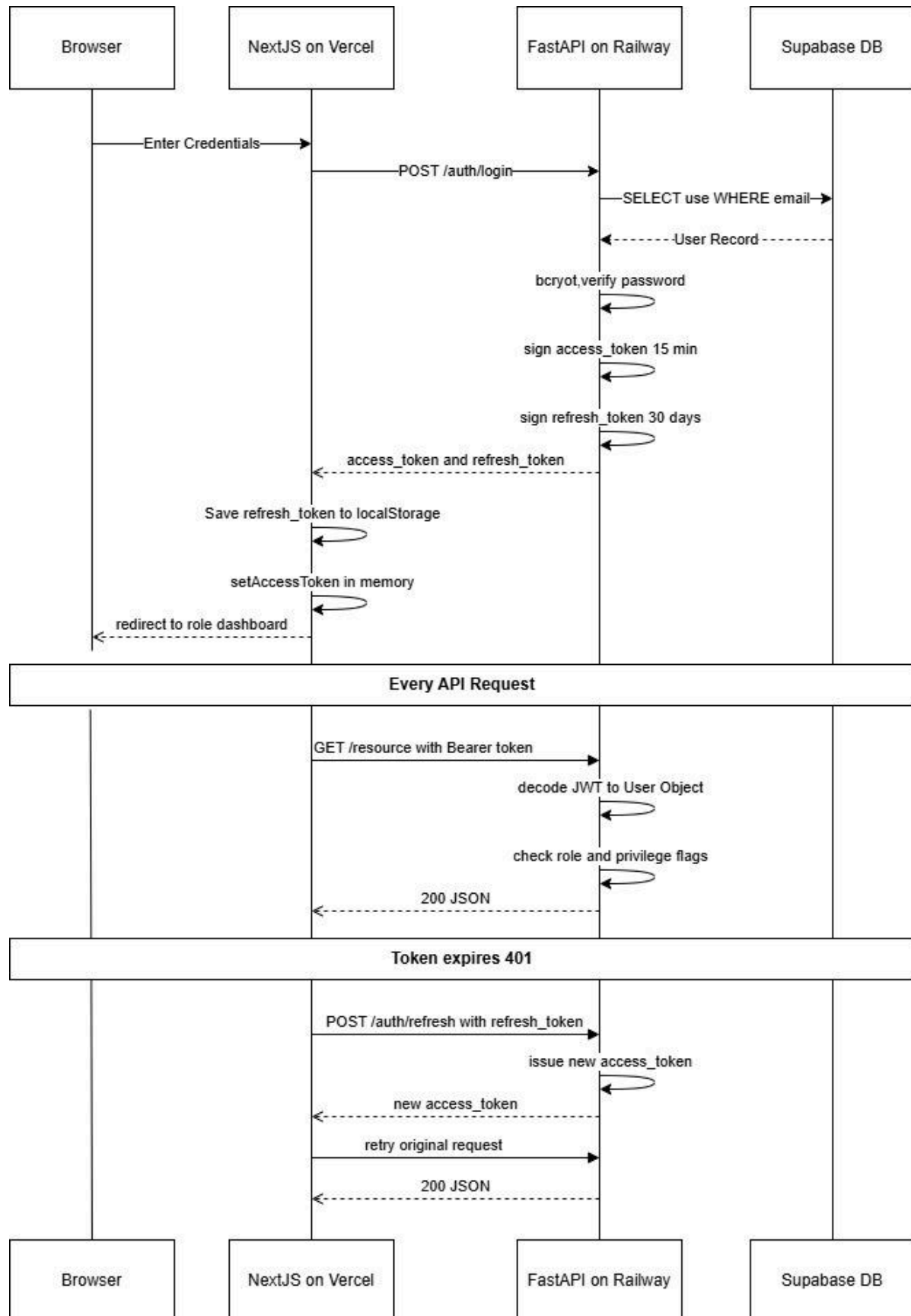
- **In-scope:** Student enrollment, attendance tracking, grade management, timetable scheduling, and a simple internal messaging system.
- **Out-of-scope:** Online fee payment gateway integration, school-bus GPS tracking, or hosteling management.

Methodology

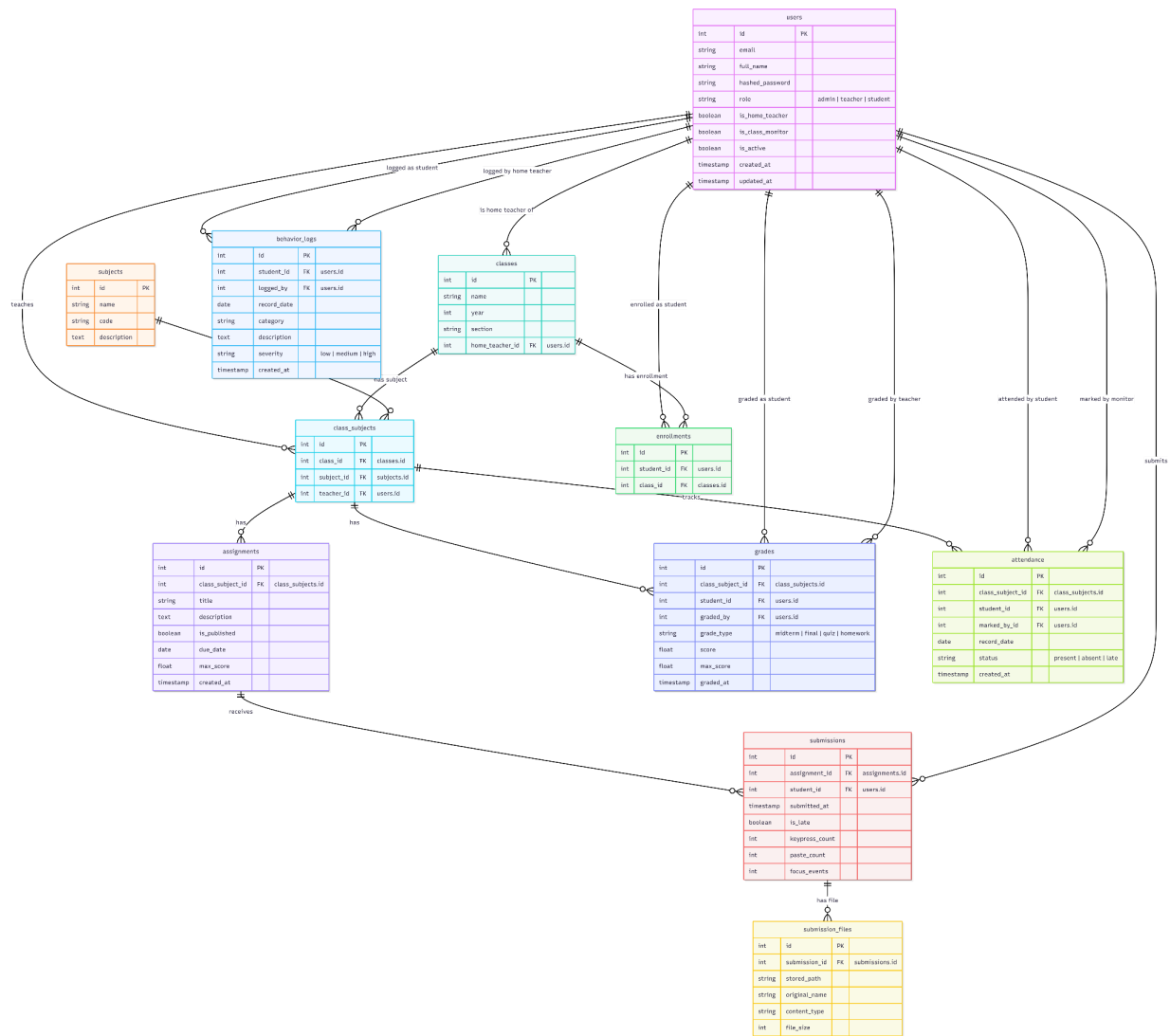
1. **Development Methodology:** Agile. We will work in a one-month sprint to allow for iterative testing and feedback [4].
2. **Technologies/Tools:**
 - a. **Frontend:** Next.js [5]
 - b. **Backend:** FastAPI [6]
 - c. **Databases:** PostgreSQL [7]
 - d. **Deployment:** Vercel [8], Supabase [9]
3. **System Architecture:**
 - a. **High-Level System Overview Diagram**



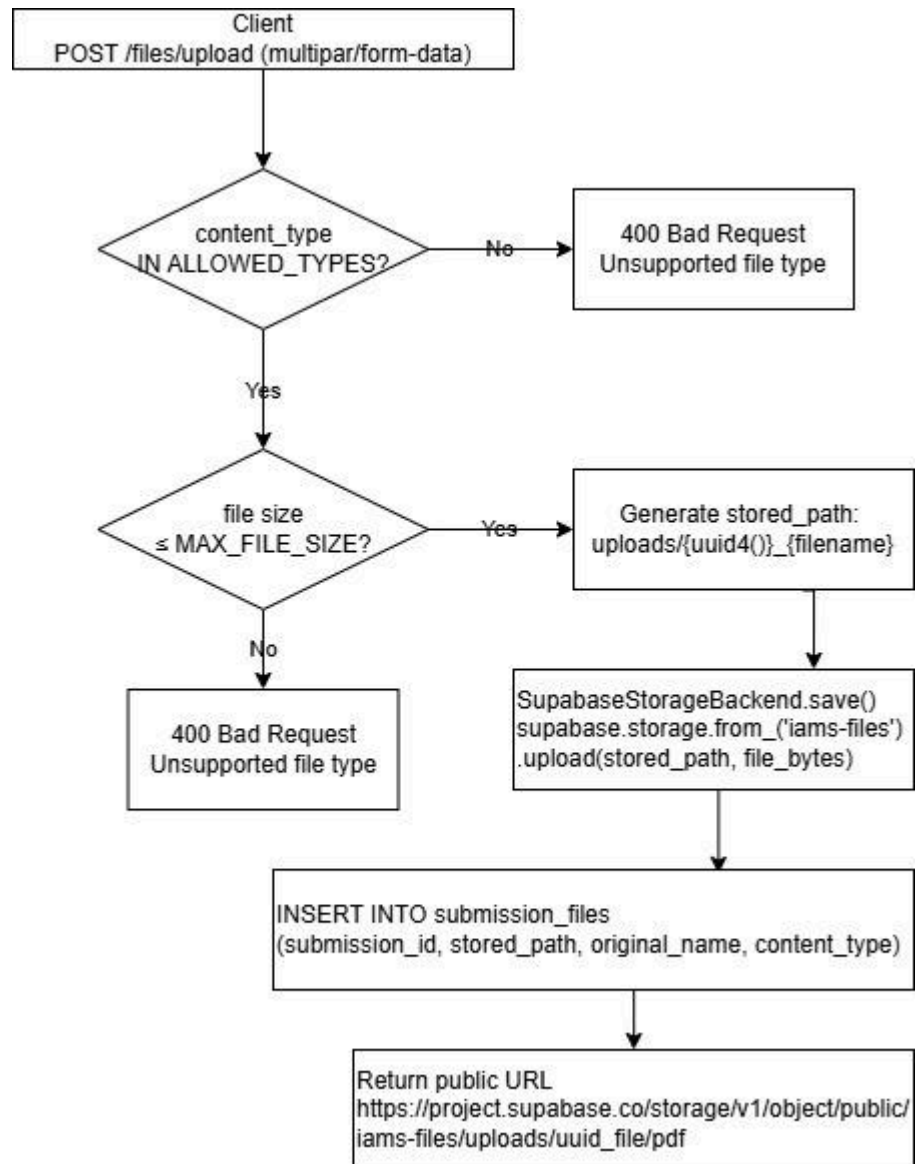
b. Authentication & Token Flow



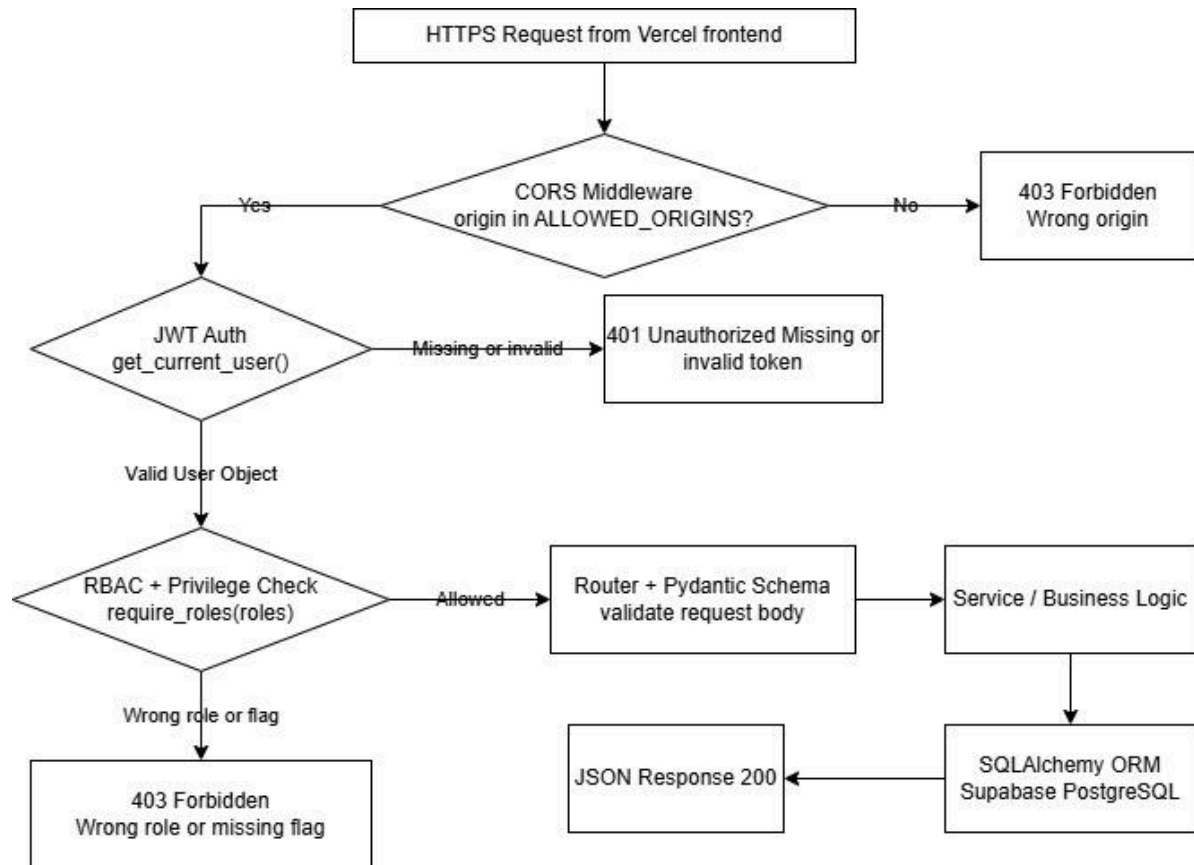
c. Database ERD (Supabase PostgreSQL)



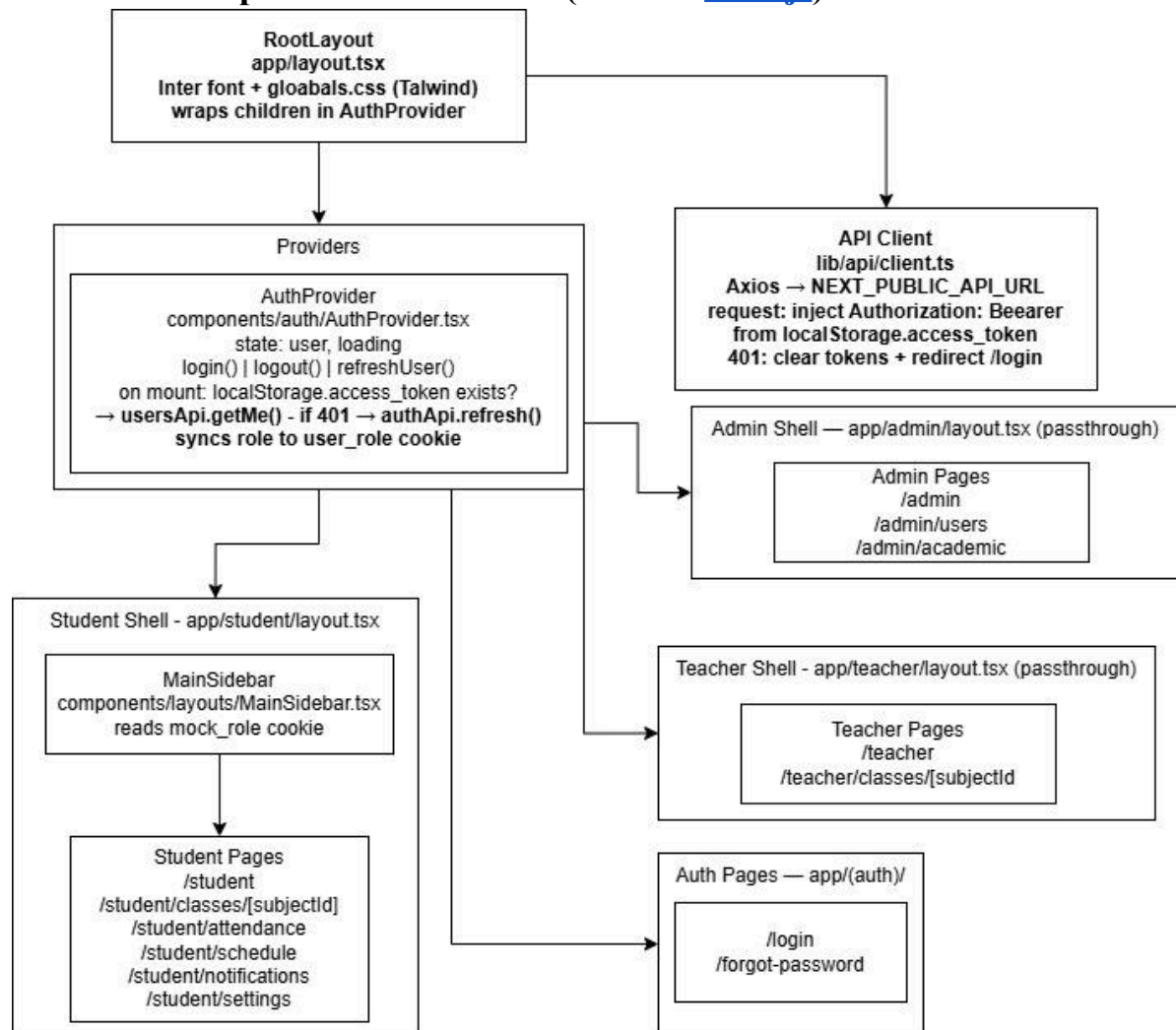
d. Supabase Storage Flow



e. Request Lifecycle Flowchart



f. Frontend Component Architecture (Vercel / [Next.js](#))



g. Environment Variables (Production)

.env.example:

FastAPI host (Railway / Render):

DATABASE_URL	postgresql://postgres.<ref>:<pass>@aws-0-<region>.pooler.supabase.com:5432/postgres
JWT_SECRET_KEY	<256-bit random secret>
STORAGE_BACKEND	supabase
SUPABASE_URL	https://<project-ref>.supabase.co
SUPABASE_SERVICE_ROLE_KEY	<service_role_key from Supabase dashboard>
SUPABASE_STORAGE_BUCKET	iams-files
ALLOWED_ORIGINS	https://your-app.vercel.app
DEBUG	false

ENVIRONMENT	production
-------------	------------

Next.js (Vercel):

NEXT_PUBLIC_API_URL	https://your-fastapi.railway.app
---------------------	----------------------------------

4. Algorithms/Models:

a. Authentication Algorithm (bcrypt + JWT)

i. Password hashing:

Registration:

raw_password → passlib.bcrypt.hash(password) → stored in users.hashed_password

Login:

passlib.bcrypt.verify(raw_password, hashed_password) → True / False

ii. JWT generation:

```
# Claims included in BOTH access and refresh tokens:
token_claims = {
    "sub": str(user.id),
    "role": user.role.value,      # "admin" | "teacher" | "student"
    "is_home_teacher": bool,     # privilege flag
    "is_class_monitor": bool,    # privilege flag
}
```

```
access_token payload = token_claims + {"exp": now +
timedelta(minutes=15), "type": "access"}
refresh_token payload = token_claims + {"exp": now +
timedelta(days=30), "type": "refresh"}
```

```
token = jose.jwt.encode(payload, JWT_SECRET_KEY,
algorithm="HS256")
```

iii. Token validation:

```
payload = jose.jwt.decode(token, JWT_SECRET_KEY,
algorithms=["HS256"])
# Returns {} on JWTError — treated as invalid
```

```
user_id = payload.get("sub")
token_type = payload.get("type")
```

```
if not user_id or token_type != "access":
    raise HTTP 401 Unauthorized
```

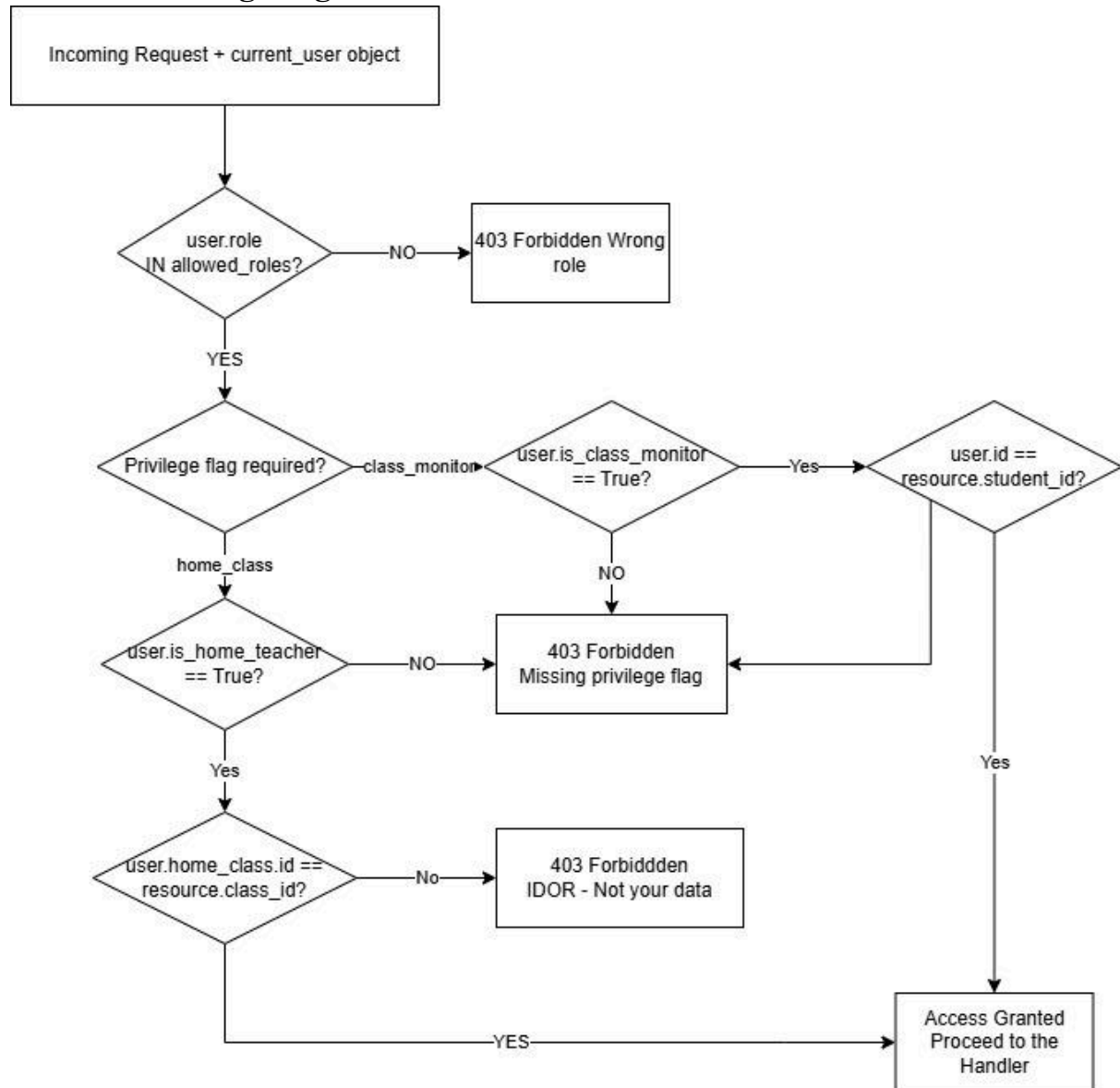
```

user = db.query(User).filter(User.id == user_id, User.is_active ==
True).first()
if not user:
    raise HTTP 401 Unauthorized

return user # user.is_home_teacher / user.is_class_monitor
available for RBAC

```

b. RBAC + Privilege Algorithm

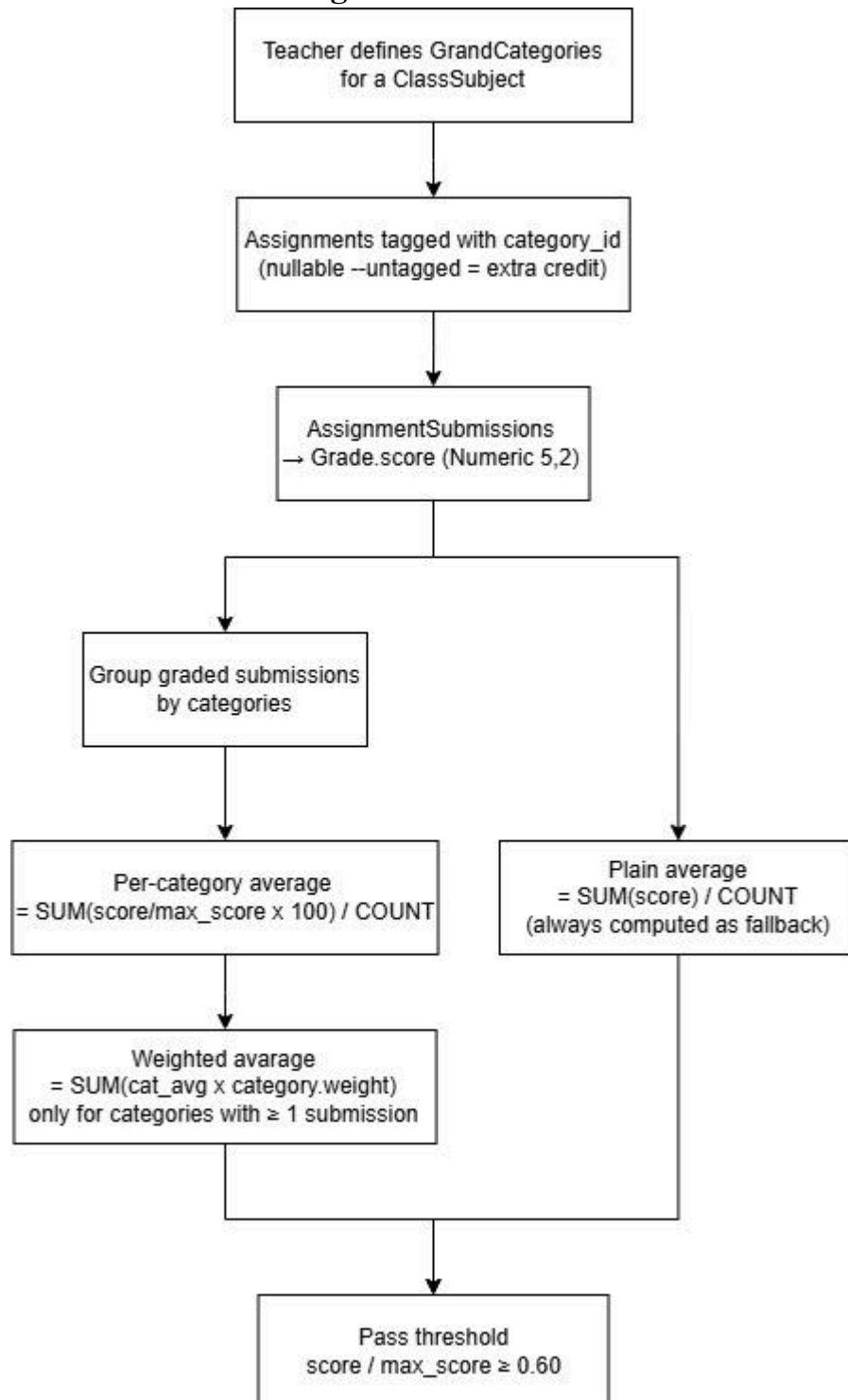


Permissions table:

Endpoint	admin	teacher	teacher + is_home_teacher	student	student + is_class_monitor
POST /classes	1	0	0	0	0

GET /classes	1	1	1	1	1
POST /attendance	0	0	0	0	1
POST /behavior_logs	0	0	1	0	0
Get /analytics	1	1	1	0	0

c. Grade Calculation Algorithm



Algorithm (Python):

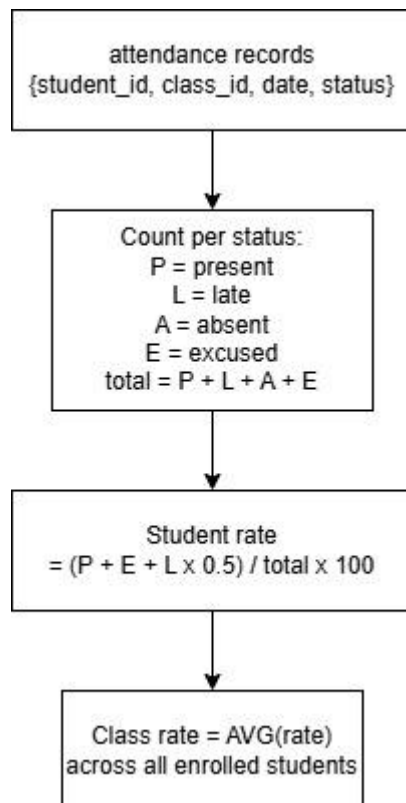
```
# In GET /analytics/student/{id}/score-trend
by_cat = defaultdict(list)
for grade, submission, assignment in rows:
    if assignment.category_id and assignment.category_id in cat_map:
        pct = grade.score / assignment.max_score * 100
        by_cat[assignment.category_id].append(pct)

weighted_average = sum(
    (sum(scores) / len(scores)) * float(cat_map[cat_id].weight)
    for cat_id, scores in by_cat.items()
) # None if no categorised submissions

plain_average = sum(grade.score for ...) / count
```

d. Attendance Rate Algorithm

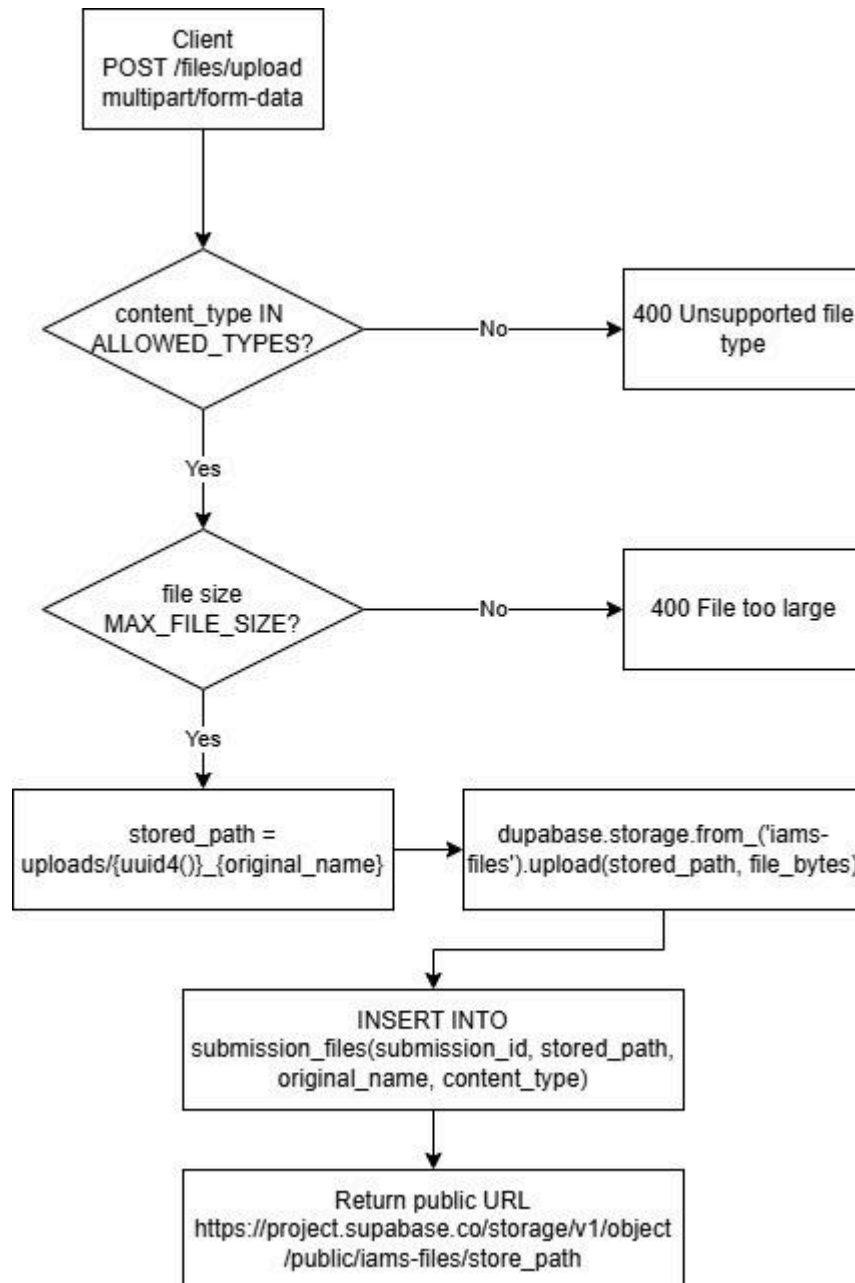
The AttendanceStatus enum has four values: present, late, absent, excused.



Formula:

$$\text{attendance_rate} = (\text{present} + \text{excused} + \text{late} \times 0.5) / \text{total_records} \times 100$$

e. File Upload Algorithm (Supabase Storage)



ALLOWED_TYPES (ALLOWED_TYPES set in files.py):

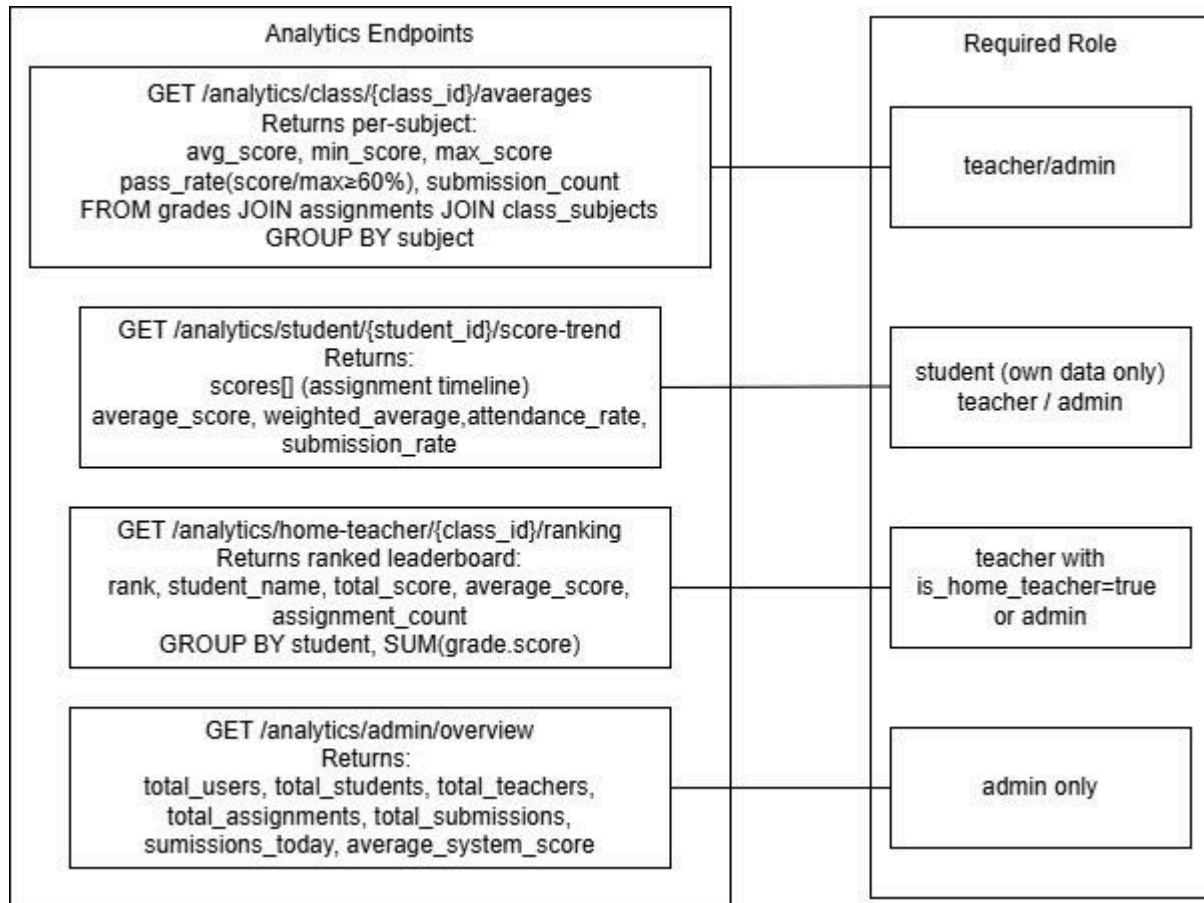
```

image/jpeg
image/png
image/gif
text/plain
application/pdf
application/msword (.doc)
application/vnd.openxmlformats-officedocument.wordprocessingml.
document (.docx)
application/vnd.openxmlformats-officedocument.spreadsheetml.she
et (.xlsx)
  
```

Storage backend selection (STORAGE_BACKEND env var):

STORAGE_BACKEND=local → LocalFilesystemBackend (default, dev)
 STORAGE_BACKEND=supabase → SupabaseStorageBackend (production)

f. Analytics Aggregation



Requirements

- **Functional Requirements:**
 - **Admin:** Manage user accounts, assign classes to teachers.
 - **Teacher:** Mark attendance, input exam scores.
 - **Student:** View grades and download schedules.
- **Non-Functional Requirements:**
 - **Security:** Password encryption and Role-Based Access Control (RBAC).
 - **Scalability:** Ability to handle records for up to 2,000 students.

Timeline

Phase	Tasks	Duration	Deadline
Research & Planning	Literature review, requirement analysis	2 weeks	26/01/2026
Design	System architecture, UI/UX design	2 weeks	09/02/2026
Implementation	Coding, Integration, Testing	6 weeks	23/03/2026
Testing & Debugging	Unit testing, System testing	2 weeks	06/04/2026
Documentation & Final Report	Writing project documentation	2 weeks	20/04/2026

Expected Outcomes

- A functional web-based School Management System.
- A comprehensive User Manual for Teachers and Admins.
- A Database Schema Design document.

Challenges and Risks

- **Technical Challenges:**
 - **Data Security:** Protecting sensitive student information.
Mitigation: Use JWT for authentication and hashed passwords.
 - **Concurrency:** Multiple teachers entering grades simultaneously.
Mitigation: Implement database locking and efficient transaction management.
- **Resource Constraints:**
 - **Data Realism:** Gaining access to actual, anonymized high school students' records for testing.
Mitigation: Use [Faker.js](#) or python Faker library to generate a realistic synthetic dataset of 2000+ students.
 - **Development Time:** The 1-month agile sprint is tight for a full-stack system with RBAC.

Mitigation: Prioritize “Must-Have” features (Enrollment, Grade) over “Nice-to-Have” (Analytic, Internal Messaging)

Budget and Resources

- **Hardware:** Laptops
- **Software:** VS Code, PostgreSQL
- **Other Resources:** Vercel, [Next.js](https://nextjs.org/), Python

References

- [1] N. C. Nweke and O. C. Ezenwa, "The Design and Implementation of Student Academic Record Management System," *ResearchGate*, 2014. [Online]. Available: https://www.researchgate.net/publication/266243197_The_Design_and_Implementation_of_Student_Academic_Record_Management_System. [Accessed: Mar. 5, 2026].
- [2] O. A. Adaramola et al., "Development of an Improved School Information Management System," *ResearchGate*, Jun. 2022. [Online]. Available: https://www.researchgate.net/publication/361017513_DEVELOPMENT_OF_A_N_IMPROVED_SCHOOL_INFORMATION_MANAGEMENT_SYSTEM. [Accessed: Mar. 5, 2026].
- [3] J. Dela Cruz et al., "Intellishool: A Student Information System for Senior High School," *ResearchGate*, 2024. [Online]. Available: https://www.researchgate.net/publication/379716395_Intellishool_A_Student_Information_System_for_Senior_High_School. [Accessed: Mar. 5, 2026].
- [4] K. Beck et al., "Manifesto for Agile Software Development," *Agile Alliance*, 2001. [Online]. Available: <http://agilemanifesto.org/>. [Accessed: Mar. 5, 2026].
- [5] Vercel, "Next.js Documentation," *Next.js*. [Online]. Available: <https://nextjs.org/docs>. [Accessed: Mar. 5, 2026].
- [6] S. Ramírez, "FastAPI," *FastAPI*. [Online]. Available: <https://fastapi.tiangolo.com/>. [Accessed: Mar. 5, 2026].

[7] PostgreSQL Global Development Group, "PostgreSQL: Documentation," *PostgreSQL*. [Online]. Available: <https://www.postgresql.org/docs/>. [Accessed: Mar. 5, 2026].

[8] Vercel, "Vercel Documentation," *Vercel*. [Online]. Available: <https://vercel.com/docs>. [Accessed: Mar. 6, 2026].

[9] Supabase, "Supabase Documentation," *Supabase*. [Online]. Available: <https://supabase.com/docs>. [Accessed: Mar. 6, 2026].

Team Member Responsibilities

Team Members	Responsibilities	Key Tasks
MENH Chanheng	Backend Lead	Database Design, System Architecture Design, API development, Security
PIN Sovanda	Frontend Lead	UI/UX design, Client-side logic, Dashboard Integration
HENG Shito	QA, Documentations	Support Frontend, System Testing. Writing the report, User Manual